

B1 — Diagrama cazurilor de utilizare

Identifici actorii, cazurile de utilizare și relațiile dintre ele

Ce sunt actorii?

Un **actor** = rol jucat de o entitate *externă* sistemului care interacționează cu el. Poate fi: persoană, alt sistem, dispozitiv hardware.

Ex: la un restaurant → Waiter , Cook , Manager . La un ATM → Client , SistemBancar .

Ce sunt cazurile de utilizare?

Un **caz de utilizare (CU)** = mulțimea tuturor scenariilor cu același obiectiv final al actorului. Reprezintă o funcționalitate oferită de sistem.

⚠ Numele unui CU trebuie să fie o expresie verbală! PlasareComandă ✓, NU Comandă ✗

Pași pentru B1

- **1. Citești enunțul** și subliniezi entitățile externe (cine interacționează cu sistemul)
- **2. Identifici actorii** — de obicei roluri de utilizatori sau sisteme externe
- **3. Identifici funcționalitățile** — ce poate face fiecare actor cu sistemul?
- **4. Desenezi diagrama** — dreptunghi (sistem) + omulețe (actori) + elipse (CU-uri) + linii de comunicare
- **5. Adaugi relații** — include / extend acolo unde are sens

Relații între CU-uri

<<include>>

CU de bază *întotdeauna* include CU-ul inclus.
CU-ul inclus nu are sens de sine stătător.
Săgeata pleacă **de la** cel de bază.

RetragereNumerar → ValidareCard

<<extend>>

CU de bază e complet și fără extindere. CU-ul care extinde e opțional/exceptional. Săgeata pleacă **de la** cel care extinde.

AccesareHELP → DepunereNumerar

Greșeli frecvente

- Actorii puși *înăuntrul* frontierei sistemului (trebuie să fie afară!)
- CU denumit cu substantiv simplu: "Comandă" în loc de "PlasareComandă"
- Săgeata de include/extend pusă în sens greșit
- Relație de comunicare între doi actori (nu are voie direct)

B2 — Descriere CU + Prototip UI

Descriri narativ cazul de utilizare cerut și schițezi interfața grafică

Șablonul de descriere tabelară

Nume	NumeleCazuluiDeUtilizare
Actori participanți	Inițiat de ActorX; comunică cu ActorY
Flux de evenimente (scenariu normal)	1. Actorul face X 2. Sistemul face Y 3. ...
Scenarii alternative	3a. Dacă Z → sistemul face W
Condiții de intrare	Ce trebuie să fie adevărat înainte
Condiții de ieșire	Ce e adevărat după finalizare
Cerințe de calitate	Performanță, timp de răspuns, etc.

Reguli la flux de evenimente

- Pașii impari = actorul face ceva; pașii pari = sistemul răspunde
- Fiecare pas trebuie să aibă o cauză clară din pasul anterior
- Nu se menționează componente concrete de UI (nu "apasă butonul X")
- Fluxul normal descrie tranzacția completă, de la inițiere la confirmare finală

Prototipul UI

O schiță simplă (box wireframe) care arată ce vede utilizatorul pe ecran în momentele cheie ale CU-ului. Nu trebuie să fie frumoasă — trebuie să arate:

- Câmpurile de input relevante
- Butoanele principale
- Mesajele de feedback (confirmare/eroare)

✓ O schiță cu dreptunghiuri + etichete e suficientă. Nu se cere design real.

B3 — Model conceptual (diagrama de clase)

Identifici conceptele din domeniu + attributele + relațiile dintre ele

Ce este modelul conceptual?

Describe conceptele din *domeniul problemei* (nu soluția software). Clasele = entități din lumea reală. Fără metode deocamdată (sau cu puține, dacă sunt evidente din enunț).

Pașii pentru B3

- **1. Identifici substantivele** din enunț → candidați la clase
- **2. Filtrezi** — elimini duplicatele, attributele greșit puse ca clase, etc.
- **3. Identifici attributele** — ce date caracterizează fiecare clasă?
- **4. Identifici relațiile** — cum sunt legate clasele între ele?
- **5. Stabilești multiplicitățile** — 1, 0..1, 1..*, 0..*, n

Tipuri de relații

Asociere

Relație generală între clase. Linie continuă + multiplicități + eventual nume de roluri.

Student — Curs (un student urmează mai multe cursuri)

Agregare / Compunere

Agregare ($\diamond$) = "are" — părțile pot exista independent. Compunere ($\blacklozenge$) = "este compus din" — părțile nu există fără întreg.

Pachet $\blacklozenge$ — CursOptional

Generalizare

Relație de moștenire. Săgeată goală de la subclasă spre superclasă.

Waiter —▷ Angajat

Dependență

Linie punctată. Folosit la proiectare (B5), mai rar la conceptual.

Multiplicități — cum le citești

1 — exact unul | 0..1 — zero sau unul | 1..* — cel puțin unul | 0..* sau * — oricâți

⚠ Multiplicitățile se citesc "de cealaltă parte": dacă pe capătul dinspre Profesor scrie 1, înseamnă că un CursOptional are exact 1 Profesor.

Greșeli frecvente

- Clase fără attribute (posibil că e un atribut al altei clase)
- Relații cu multiplicități greșite (citite de pe mauvais capăt)
- Lipsa enumerărilor (dacă enunțul menționează stări/categorii fixe → «enumeration»)
- Metode puse la B3 (rămân pentru B5)



B4 — Diagrama de secvență

Arăți cum interacționează obiectele pentru scenariul normal al CU-ului

Structura diagramei

Coloane = lifelines (linii de viață ale obiectelor). Timp curge de sus în jos. Mesajele = săgeți orizontale între lifelines.

Ordinea standard a coloanelor:

1. Actor (inițiator)
2. Obiect Boundary (interfața cu actorul — formular, buton, ecran)
3. Obiect Control (gestionează CU-ul)
4. Obiecte Entity (datele persistente)

Clase Boundary / Control / Entity

«boundary»

Interfața cu actorul. Ex: `FormularLogin` , `EcranComanda` . Creat de actor prin interacțiune.

«control»

Gestionează logica CU-ului. Ex: `ControlLogin` . Creat de Boundary, distrus la final. Un control per CU (regulă generală).

«entity»

Datele persistente. Ex: `Waiter` , `Order` . Accesate de Boundary și Control, dar **niciodată** nu accesează Boundary sau Control.

Reguli cheie

- Dreptunghiul de activare (execution box) apare când un obiect execută o metodă
- Mesajele pleacă din dreptunghiul de activare al obiectului care apelează
- Mesajul de return (linie punctată) marchează sfârșitul execuției
- Obiectele Control sunt create de Boundary cu mesaj «create»
- Obiectele Boundary noi sunt create de Control, nu de actor direct
- Numerotare imbricată opțională: 1, 1.1, 1.2, 2, 2.1...

Cum traduci pașii din CU în mesaje

Pas CU: "1. Waiter-ul selectează masa și apasă Preia Comandă"

→ Actor → Boundary: `selectMasa(tableNo)`

Pas CU: "2. Sistemul afișează formularul de comandă"

→ Boundary → Control: `initComanda()` → Control creează obiectele necesare → returnează date spre Boundary

Greșeli frecvente

- Lipsa obiectului Control (sărit direct la Entity)
- Entity care trimite mesaje la Boundary (interzis!)
- Dreptunghiuri de activare care nu se închid corect (nested greșit)
- Mesajele între Actor și Entity direct (trebuie să treacă prin Boundary + Control)



B5 — Diagrama detaliată de clase (proiectare)

Rafinezi modelul conceptual + adaugi clasele noi din B4 + completezi metode + relații

Ce adaugi față de B3?

- **Clasele noi identificate la B4:** Boundary + Control (cu stereotipurile lor)
- **Metodele** pentru fiecare clasă (derivate din mesajele din B4)
- **Atributele complete** cu tipuri
- **Relațiile de dependență** (linie punctată) acolo unde un obiect folosește altul ca parametru/variabilă locală
- **Navigabilitatea** asocierilor (săgeți unidirecționale unde e cazul)

Regula de bază: mesaj → metodă + relație

Dacă în B4 obiectul `o1:C1` trimite mesaj `op()` la `o2:C2` :

→ Clasa `C2` trebuie să aibă metoda `op()`

→ Există o relație **asociere** sau **dependență** de la `C1` la `C2`

Asociere vs Dependență

Asociere

Dacă `C1` păstrează o referință persistentă la `C2` (câmp membru). Comunică și în alte contexte.

Dependență

Dacă `C1` folosește `C2` doar temporar (parametru, variabilă locală). Linie punctată cu săgeată.

Stereotipuri pentru clasele noi

«boundary» `FormularComanda` — interfață cu utilizatorul
«control» `ControlComanda` — logica cazului de utilizare
«entity» `Order` — date persistente (de obicei deja în B3)

Checklist final B5

- ☐ Toate clasele din B4 sunt prezente (inclusiv Boundary + Control)
- ☐ Fiecare mesaj din B4 → metodă în clasa destinatară
- ☐ Atribute cu tip specificate
- ☐ Multiplicități pe toate asocierile
- ☐ Navigabilitate indicată (săgeți unidirecționale unde comunicarea e într-un singur sens)
- ☐ Enumerările din B3 păstrate